

Detector-side Program 1 (Pitch Angle)

```
// This program was written by Hiroyuki Narusawa

#include <stdlib.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <Preferences.h>
#include "MPU9250.h"
#include <MadgwickAHRS.h>
#define SU "0000ffff-0000-1000-8000-00805f9b34fb" //Service UUID
#define CU "beb5483e-36e1-4688-b7f5-ea07361b26a8" //Characteristic UUID

Madgwick filter;
// unsigned long microsPerReading, microsPrevious;
// float accelScale, gyroScale;
String msg = "00";
char txmsg[10] = "0000000000";
float offset=0;
int mult=1;
float tx_v=0;
int tmp_ad=0;
Preferences preferences;
BLEService *pService;//Declare a service
BLEAdvertising *pAdvertising;//Declare advertising
BLECharacteristic *pCharacteristic;//Declare a characterisitc

// an MPU9250 object with the MPU-9250 sensor on I2C bus 0 with address 0x68
MPU9250 IMU(Wire,0x68);
int status;

void setup() {

  pinMode(18,INPUT_PULLUP);
  pinMode(19,INPUT_PULLUP);
  Serial.begin(115200);
  filter.begin(25);
  while(!Serial) {}
  preferences.begin("pedal",false); //start reading the eeprom
  mult=preferences.getInt("h",1); //read an eeprom factor (if not, set it to 1)
  preferences.end(); //End the eeprom

  //-----
  BLEDevice::init("MASTER"); //Initialize a BLE device
  BLEServer *pServer = BLEDevice::createServer(); //Set it as pServer
  pService = pServer->createService(SU); //Construct the service by using a pServer member function createService
  //Set a characteristic by using a pService member function create Characteristic and set it by using (READ and WRITE)
  pCharacteristic = pService->createCharacteristic(
    CU,BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE);
  pCharacteristic->setValue(msg,c_str()); //pCharacteristic: call a setValue member function
  pService->start(); //pService: call a start member function and start it

  //Give pAdvertising a return value ad the pServer getAdvertising member function
  pAdvertising = pServer->getAdvertising();
  //Use a return value of a pService getUUID member function as an argument and give the argument to a pAdvertising addServiceUUID member function
  pAdvertising->addServiceUUID(pService->getUUID());
  pAdvertising->start(); //Call a pAdvertising start member function

  // start communication with IMU
  status = IMU.begin();
  if (status < 0) {
    Serial.println("IMU initialization unsuccessful");
    Serial.println("Check IMU wiring or try cycling power");
    Serial.print("Status: ");
    Serial.println(status);
    while(1) {}
  }
  // setting the accelerometer full scale range to +/-8g
  IMU.setAccelRange(MPU9250::ACCEL_RANGE_8G);
  // setting the gyroscope full scale range to +/-500 deg/s
  IMU.setGyroRange(MPU9250::GYRO_RANGE_2000DPS);
  // setting DLPF bandwidth to 20 Hz
  IMU.setDlpfBandwidth(MPU9250::DLPF_BANDWIDTH_184HZ);
  // setting SRD to 19 for a 50 Hz update rate
  IMU.setSrd(1);

  void loop() {
    / int aIx, aIy, aIz;
    / int gIx, gIy, gIz;
    float ax, ay, az;
    float gx, gy, gz;
    float mx, my, mz;
    float roll, pitch, heading;
    / unsigned long microsNow;
    // read the sensor
    IMU.readSensor();
    ax=IMU.getAccelX_mss();
    ay=IMU.getAccelY_mss();
    az=IMU.getAccelZ_mss();
    gx=IMU.getGyroX_rads();
    gy=IMU.getGyroY_rads();
    gz=IMU.getGyroZ_rads();
    mx=IMU.getMagX_uT();
    my=IMU.getMagY_uT();
    mz=IMU.getMagZ_uT();
    filter.update(gx, gy, gz, ax, ay, az,mx,my,mz);
    roll = filter.getRoll();
    pitch = filter.getPitch();
    heading = filter.getYaw();
    tx_v=pitch; // set tx_v as a pitch angle
    tx_v+=offset; //+ offset
    if(tx_v<0) tx_v=0; //if the pitch angle is negative, set it to 0
    tx_v=tx_v*mult; //multiply the pitch angle by a multiplier
    if(tx_v>99) tx_v=99; //if the tx_v exceeds 99, it is set to 99
    itoa((int)tx_v,txmsg,10); //The tx_v is converted from a float to an integer, and it is then converted to a string
    pCharacteristic->setValue(txmsg); // call a pCharacteristic setValue member function

    / Serial.print("Y:");Serial.print(heading);
    Serial.print(" P:"); Serial.print(pitch);
    / Serial.print(" R:"); Serial.print(roll);
    Serial.print(" 0:"); Serial.print(offset); Serial.print(" M:"); Serial.print(mult);
    Serial.print(" TX:");Serial.println(txmsg);

    if (!digitalRead(19)) { //if the left button is off, the following is executed.
      tmp_ad=analogRead(A4); // gpio32
      offset=map(tmp_ad,0,4095,-45,45 );
    }
    if (!digitalRead(18)) { //if the right button is off
      tmp_ad=analogRead(A5); // gpio33
      mult=map(tmp_ad,0,4095,1,50);
      preferences.begin("pedal",false); // store a multiplier
      preferences.putInt("h", mult);
      preferences.end();
    }

    delay(1);
  }
}
```

Actuator-side Program 1 (Pitch Angle)

```
// This program was written by Hiroyuki Narusawa

#include <Preferences.h> //

// BLE client M5STACK      Operate as a BLUETOOTH LE client
#include "BLEDevice.h"
#include <string.h>
#define Ready 34           // Set Ready to D34
#define Mo0 32             // Set Mo0 to D32
#define Mo1 33             // Set Mo1 to D33
#define Mo2 25             // Set Mo2 to D25
#define Mo3 26             // Set Mo3 to D26
#define Mo4 27             // Set Mo4 to D27
#define Mo5 14             // Set Mo5 to D14
#define Start 13           // Set Start to D13
#define SETUP_TOP_SW 35    // Set SETUP_TOP_SW to 35
#define SETUP_W_SW 39      // Set an actuation range value SETUP_W_SW to D39

Preferences preferences;   // Define a READ|WRITE class of a non-volatile memory

union {                    // Define a union so as to convert a string to bits
  unsigned char chr;
  struct bits {
    unsigned char b1 : 1;
    unsigned char b2 : 1;
    unsigned char b3 : 1;
    unsigned char b4 : 1;
    unsigned char b5 : 1;
    unsigned char b6 : 1;
    unsigned char b7 : 1;
    unsigned char b8 : 1;
  } b;
};

char ss0[10]="000000000";
char ss1[10]="000000000";
// Direct I/O control for an Oriental motor AR/AZ

int w_ad=0;              //
int top_ad=0;            //
int set_pos=0;           //
int top_tmp=0;           //
int w_tmp=0;             //
int def=0;
static BLEUUID SU ("0000ffff-0000-1000-8000-00005f9b34fb"); // Service UUID
static BLEUUID CU ("beb5483e-36e1-4688-b7f5-ea07361b26a8"); //Characteristic UUID
static BLEAddress *pServerAddress; //Declare a server address pointer function
static boolean doConnect = false; //
static boolean connected = false; //
static BLERemoteCharacteristic* pRemoteCharacteristic; //Declare a remote characteristic point function
//=====Receive a notification (data) from a server=====
static void notifyCallback(BLERemoteCharacteristic* pBLERemoteCharacteristic,uint8_t* pData,size_t length,bool isNotify) {pBLERemoteCharacteristic->getUUID().toString().c_str();}
//=====Server connection function=====
bool connectToServer(BLEAddress pAddress) {
  BLEClient* pClient = BLEDevice::createClient();
  pClient->connect(pAddress);
  BLERemoteService* pRemoteService = pClient->getService(SU);
  if (pRemoteService == nullptr) {
    return false;
  }
  pRemoteCharacteristic = pRemoteService->getCharacteristic(CU);
  if (pRemoteCharacteristic == nullptr) {
    return false;
  }
  std::string value = pRemoteCharacteristic->readValue();
  pRemoteCharacteristic->registerForNotify(notifyCallback);
}
//=====Advertising call back function=====
class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks{
  void onResult(BLEAdvertisedDevice advertisedDevice) {
    if (advertisedDevice.haveServiceUUID() &&
        advertisedDevice.getServiceUUID().equals(SU)) {
      advertisedDevice.getScan()->stop();
      pServerAddress = new BLEAddress(advertisedDevice.getAddress());
      doConnect = true; // if the UUID is matched, set TRUE
    }
  }
};

// Connect Ready, M0, M1, M2, Start, and Stop to controller's assigned terminals
// The assignment and operation is set by a MEXE02 software of Oriental Motor
// mov(0,0,0,0,0,0) No.0 : return to the original point or an actuation start point
// mov(0,0,0,0,0,1) No.48 Full operation
// mov(0,0,0,0,1,0) No.49 Half operation
// mov(0,0,0,0,1,1) No.50 Return operation

void mov(void){
  while(digitalRead(Ready)==LOW); // Wait for completion of the previous processing
  digitalWrite(Mo0,u.b.b1);      // Mo0, Mo1, Mo2, Mo3, Mo4, and Mo5 are used tp set the operation Number.
  digitalWrite(Mo1,u.b.b2);      //
  digitalWrite(Mo2,u.b.b3);      //
  digitalWrite(Mo3,u.b.b4);      //
  digitalWrite(Mo4,u.b.b5);      //
  digitalWrite(Mo5,u.b.b6);      //
  delay(5);                      // to avoid a malfunction
  digitalWrite(Start,1);         // After the operation is set, execute the start operation
  while(digitalRead(Ready)==HIGH);
  digitalWrite(Start,0);        //
}

//=====
void setup() {

  Serial.begin(9600);
  pinMode(Ready,INPUT_PULLUP); // Define a BLE scan
  pinMode(SETUP_TOP_SW,INPUT); //
  pinMode(SETUP_W_SW,INPUT); //

  pinMode(Mo0,OUTPUT); //
  pinMode(Mo1,OUTPUT); //
  pinMode(Mo2,OUTPUT); //
  pinMode(Mo3,OUTPUT); //
  pinMode(Mo4,OUTPUT); //
  pinMode(Mo5,OUTPUT); //
  pinMode(Start,OUTPUT); //

  preferences.begin("pedal",false); //Start reading and writing data in the EEPROM
  top_ad=preferences.getInt("top",0); //Read EEPROM top data (if there is no data, set it to 0)
  w_ad=preferences.getInt("w",47); //Read an EEPROM actuation range value (if there is no data, set it to 47)
  preferences.end(); //End the EEPROM

  u.chr=(unsigned char)top_ad; //move a pedal to the top position
  mov();

  BLEDevice::init(""); //Initialize the Bluetooth
  BLEScan* pBLESscan = BLEDevice::getScan();
  pBLESscan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks()); //Set an advertising call back function
  pBLESscan->setActiveScan(true); //Make a scan effective
  pBLESscan->start(30); //Start the scan
}

//=====
void loop() {
  if (doConnect == true) { //If the advertising result is TRUE
    if (connectToServer(*pServerAddress)) { //Connect to the server
      connected = true;
    }
    doConnect = false; //Limit the server connection only once
  }
  if (connected) { //
    std::string value = pRemoteCharacteristic->readValue(); //Read notification data
    strcpy(ss0,value.c_str());
    if (digitalRead(SETUP_TOP_SW)==1) && (digitalRead(SETUP_W_SW)==1) { // if a setting adjuster is off
      if (strcmp(ss0,ss1)!=0) { // if data is changed
        def=atoi(ss0); // Convert a string to an integer
        strcpy(ss1,ss0); // store data
      }
    }
  }
}
```

Detector-side Program 2 (Rotation Rate)

```
// This program is written by Tomoyuki Shishido

// BLE SERVER
//=====
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEServer.h>
#include <Arduino.h>
#include <utility/MPU9250.h>
#include <utility/quaternionFilters.h>
#include <utility/Button.h>

#define processing_out false
#define AHRS true
#define SerialDebug true
#define LCD

#define SU "0000ffff-0000-1000-8000-00805f9b34fb"
#define CU "beb5483e-36e1-4688-b7f5-ed07361b26a8"
MPU9250 IMU;
String msg = "C";
String oldmsg = "C";
int thresholdValue = 30;

BLEService *pService;
BLEAdvertising *pAdvertising;
BLECharacteristic *pCharacteristic;
//=====

void setup() {
  //Serial.begin(115200);
  MS.begin();
  Wire.begin();
  MS.Lcd.fillScreen(BLACK);

  //-----
  BLEDevice::init("MASTER");
  BLEServer *pServer = BLEDevice::createServer();
  pService = pServer->createService(SU);
  pCharacteristic = pService->createCharacteristic(
    CU, BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE);
  pCharacteristic->setValue(msg.c_str());
  pService->start();
  pAdvertising = pServer->getAdvertising();
  pAdvertising->addServiceUUID(pService->getUUID());
  pAdvertising->start();

  IMU.initMPU9250();

  Serial.println("MPU9250 initialized for active data mode....");

  MS.Lcd.setTextSize(1);
  MS.Lcd.setTextColor(GREEN, BLACK);
  MS.Lcd.fillScreen(BLACK);
}
//=====

void loop() {
  // If intPin goes high, all data registers have new data
  // On interrupt, check if data ready interrupt
  if (IMU.readByte(MPU9250_ADDRESS, INT_STATUS) & 0x01)
  {
    IMU.readGyroData(IMU.gyroCount); // Read the x/y/z adc values
    IMU.getGres();

    // Calculate the gyro value into actual degrees per second
    // This depends on scale being set
    IMU.gx = (float)IMU.gyroCount[0]*IMU.gRes;
    IMU.gy = (float)IMU.gyroCount[1]*IMU.gRes;
    IMU.gz = (float)IMU.gyroCount[2]*IMU.gRes;
  }

  // Must be called before updating quaternions!
  IMU.updateTime();

  MahonyQuaternionUpdate(IMU.ax, IMU.gy, IMU.az, IMU.gx*DEG_TO_RAD,
    IMU.gy*DEG_TO_RAD, IMU.gz*DEG_TO_RAD, IMU.my,
    IMU.mx, IMU.mz, IMU.deltat);

  if (!AHRS)
  {
    IMU.deltat = millis() - IMU.count;
    if (IMU.deltat > 500)
    {
      if(SerialDebug)
      {
        // Print gyro values in degree/sec
        Serial.print("X-gyro rate: "); Serial.print(IMU.gx, 3);
        Serial.print(" degrees/sec ");
        Serial.print("Y-gyro rate: "); Serial.print(IMU.gy, 3);
        Serial.print(" degrees/sec ");
        Serial.print("Z-gyro rate: "); Serial.print(IMU.gz, 3);
        Serial.println(" degrees/sec");
      }
    }
  }

#ifdef LCD
  MS.Lcd.fillScreen(BLACK);
  MS.Lcd.setTextColor(GREEN, BLACK);
  MS.Lcd.setCursor(0, 0); MS.Lcd.print("MPU9250/AK8963");
  MS.Lcd.setCursor(0, 32); MS.Lcd.print(" x y z ");

  MS.Lcd.setCursor(0, 64); MS.Lcd.print((int)(IMU.gx));
  MS.Lcd.setCursor(32, 64); MS.Lcd.print((int)(IMU.gy));
  MS.Lcd.setCursor(64, 64); MS.Lcd.print((int)(IMU.gz));
  MS.Lcd.setCursor(96, 64); MS.Lcd.print("o/s");
#endif

  IMU.count = millis();
} // if (IMU.deltat > 500)
} // if (!AHRS)
else
{
  // Serial print and/or display at 0.5 s rate independent of data rates
  IMU.deltat = millis() - IMU.count;

  // update LCD once per half-second independent of read rate
  // if (IMU.deltat > 500)
  if (IMU.deltat > 100)
  {
    if(SerialDebug)
    {
      Serial.print("gx = "); Serial.print( IMU.gx, 2);
      Serial.print(" gy = "); Serial.print( IMU.gy, 2);
      Serial.print(" gz = "); Serial.print( IMU.gz, 2);
      Serial.println(" deg/s");
    }
  }
}

}

if(IMU.gz>thresholdValue && (oldmsg != "A")){
  msg = "A";
  oldmsg = msg;
  pCharacteristic->setValue(msg.c_str());
}
else{
  if(IMU.gz>=-thresholdValue){
    msg = oldmsg;
    pCharacteristic->setValue(msg.c_str());
  }
  else{
    msg = "C";
    oldmsg = msg;
    pCharacteristic->setValue(msg.c_str());
  }
}

#ifdef LCD
  // MS.Lcd.fillScreen(BLACK);
  MS.Lcd.setCursor(0, 0); MS.Lcd.print(" x y z ");

  MS.Lcd.setCursor(0, 44);
  MS.Lcd.printf("% 6d % 6d % 6d o/s \r\n", (int)(IMU.gx), (int)(IMU.gy), (int)(IMU.gz));

  MS.Lcd.setCursor(0, 180);
  MS.Lcd.printf("Threshold: % 6d \r\n", (thresholdValue));
  MS.Lcd.print(" Adjust - + ");
#endif

IMU.count = millis();
IMU.sumCount = 0;
IMU.sum = 0;

} // if (IMU.deltat > 500)
} // if (AHRS)

if (digitalRead(38)) {
  thresholdValue += 1;
  if (thresholdValue > 80){thresholdValue = 80;}
  delay(150);
}

if (digitalRead(37)) {
  thresholdValue -= 1;
  if (thresholdValue < 1){thresholdValue = 1;}
  delay(150);
}

}
```